

FastAPI & Backend Development

Interview Revision Handout — Section 1: Web & API Fundamentals + FastAPI Basics

PART 1 — API, REST & HTTP FUNDAMENTALS

1. What is an API?

An **API (Application Programming Interface)** is a contract that lets two software systems talk to each other. It defines what requests a client can send, and what responses it will get back — without exposing internal implementation details.

- Think of it as a **waiter** in a restaurant: you (client) don't go into the kitchen (server/database); you order via the waiter (API).
- Types: Web APIs (REST, GraphQL, SOAP), Library APIs, OS APIs, Hardware APIs.

2. What is REST?

REST (Representational State Transfer) is an architectural style (not a protocol) for designing networked applications, defined by Roy Fielding in 2000. It uses standard HTTP methods to operate on **resources** identified by URLs.

Core REST principles:

- **Statelessness** — every request contains all info needed; server stores no client session/context.
- **Client-Server separation** — frontend and backend evolve independently.
- **Uniform Interface** — consistent resource naming & standard HTTP verbs.
- **Resource-based** — everything is a resource, addressed by a URL (e.g. /users/5).
- **Cacheable** — responses define if they can be cached.
- **Layered System** — client can't tell if it's talking directly to the server or a proxy/load balancer.

3. What is a REST API?

A **REST API** is a web API that follows REST principles — it exposes resources over HTTP, using URLs as resource identifiers and HTTP methods (GET, POST, PUT, DELETE, PATCH) as the actions performed on them. Data is usually exchanged in **JSON** format.

```
GET https://api.example.com/users/5 → returns user with id 5
```

■ **Interview Q:** What's the difference between REST and RESTful? — REST is the style/set of rules; "RESTful" describes an API that actually implements/follows those rules.

4. What is an HTTP Method (Verb)?

HTTP methods indicate the **action** the client wants to perform on a resource. They tell the server the intent of the request.

Method	Purpose	Idempotent?	Safe?	Typical Body
GET	Retrieve a resource	Yes	Yes	No
POST	Create a new resource	No	No	Yes
PUT	Replace a resource fully	Yes	No	Yes
PATCH	Partially update a resource	No*	No	Yes
DELETE	Remove a resource	Yes	No	Usually no

*PATCH is not idempotent by spec, though in practice it's often implemented as idempotent.

■ **Interview Q:** Idempotent means calling it once or many times gives the same server state. GET/PUT/DELETE are idempotent; POST is not (calling it twice creates two resources).

5. Headers

HTTP Headers are key-value metadata sent with a request or response — they describe the data, not the data itself (auth info, content type, caching rules, etc.).

Header	Used In	Meaning
--------	---------	---------

Content-Type	Request/Response	Format of the body, e.g. application/json
Authorization	Request	Credentials, e.g. Bearer <token>
Accept	Request	What response format client expects
Cache-Control	Response	Caching rules for the resource
Set-Cookie	Response	Sends a cookie to store on client

6. Endpoint

An **endpoint** is a specific URL + HTTP method combination where an API can be accessed — the exact point where a client's request meets server logic.

```
GET /users      → list all users
GET /users/{id} → get one user
POST /users     → create a user
```

7. Request & Response Model

- **Request** = Method + URL + Headers + (optional) Body/Params sent by the client.
- **Response** = Status Code + Headers + Body sent back by the server.
- In FastAPI, these are represented as **Pydantic models** (Python classes) that validate incoming JSON and shape outgoing JSON automatically.

```
class UserRequest(BaseModel):
    name: str
    age: int

class UserResponse(BaseModel):
    id: int
    name: str
```

PART 2 — FASTAPI ESSENTIALS

1. What is FastAPI?

FastAPI is a modern, high-performance Python web framework for building APIs, based on standard Python type hints. It was created by **Sebastián Ramírez (tiangolo)**, first released in 2018.

2. Endpoints via Decorators

A **decorator** is a Python function that wraps another function to add extra behaviour without changing its code (uses the `@` syntax). FastAPI uses decorators like `@app.get()` to register a Python function as the handler for a specific route + method.

```
@app.get("/users/{id}")
def get_user(id: int):
    return {"id": id}
```

■ **Interview Q:** Why decorators here? — They cleanly bind a URL path + HTTP method to a plain Python function, keeping routing declarative and readable.

3. Key Advantages of FastAPI

- **Automatic Data Validation** — Powered by **Pydantic**. Type hints on function parameters are enforced at runtime; invalid data returns a 422 error automatically.
- **Automatic API Documentation** — Generates interactive **Swagger UI** (/docs) and **ReDoc** (/redoc) from your code, based on the OpenAPI standard — no manual doc writing.
- **Dependency Injection** — The `Depends()` system lets you declare reusable logic (DB sessions, auth checks, config) that FastAPI automatically resolves and injects into route functions.
- **Async Support** — Native `async def` support lets FastAPI handle many concurrent I/O-bound requests (DB calls, external APIs) efficiently on a single thread.
- **Type-hint driven** — Editor autocompletion, fewer bugs, self-documenting code.

- **High Performance** — One of the fastest Python frameworks, comparable to Node.js/Go, because it runs on Starlette + ASGI.

Dependency Injection — Quick Example

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/items")
def read_items(db: Session = Depends(get_db)):
    return db.query(Item).all()
```

4. Async Code in FastAPI

async def endpoints run on an **event loop (ASGI)** instead of blocking a thread while waiting on I/O (DB, network calls). This lets one worker handle thousands of concurrent requests. Use async def for I/O-bound work with async libraries; plain def still works fine — FastAPI runs it in a threadpool automatically.

5. Libraries FastAPI is Built On

Library	Role
Starlette	The ASGI toolkit underneath — handles routing, middleware, WebSockets, async request/response handling.
Pydantic	Handles data validation, parsing, and serialization using Python type hints.
Uvicorn	The ASGI server that actually runs the FastAPI app.

■ **Interview Q:** FastAPI = Starlette (web parts) + Pydantic (data parts), combined with an elegant developer-friendly API.

PART 3 — FASTAPI vs FLASK vs DJANGO

A frequent interview ask: "Why choose FastAPI over Flask/Django?" Use this comparison as your quick answer key.

Aspect	FastAPI	Flask	Django
Type	Async micro-framework	Sync micro-framework	Full-stack (batteries-included)
Performance	Very high (ASGI, async)	Moderate (WSGI, sync)	Moderate (WSGI, sync)
Data Validation	Built-in via Pydantic	Manual / extensions	Via Django Forms/DRF serializers
API Docs (OpenAPI)	Automatic (Swagger/ReDoc)	Manual (Flask-RESTX etc.)	Manual (via DRF + extra tools)
Async Support	Native, first-class	Limited/partial (Flask 2+)	Partial (Django 4+ ASGI views)
Dependency Injection	Built-in (Depends)	Not built-in	Not built-in
Tooling / Ecosystem	Growing, focused on APIs	Large, very flexible	Largest — admin, ORM, auth built-in
Learning Curve	Low–Medium	Low	Medium–High
Security	Relies on you + libs (OAuth2 helpers)	Relies on you + extensions	Strong built-in security (CSRF, auth, ORM)
Scalability	Excellent for async I/O-heavy APIs	Good with extra setup	Good, proven at large scale
Best Use Case	High-performance APIs, microservices	Simple apps, simple APIs, full control	Large monolith apps needing admin/auth/ORM out of the box

■ **Interview Q:** One-line answer: "FastAPI is the fastest for building async, type-safe, auto-documented APIs; Flask gives more manual control for simple apps; Django is best when you need a full ecosystem (admin panel, ORM, auth) out of the box."